Optimizing General Purpose Compiler Optimization

M. Haneda
LIACS
Leiden University
The Netherlands
haneda@liacs.nl

P.M.W. Knijnenburg
LIACS
Leiden University
The Netherlands
peterk@liacs.nl

H.A.G. Wijshoff
LIACS
Leiden University
The Netherlands
harryw@liacs.nl

ABSTRACT

The problem of defining optimal optimization strategies for compilers is well known and has been studied extensively over the last years. The problem arises from the fact that the sheer number of possible combinations of optimizations, their order, and their setting creates a search space which cannot adequately be searched. Although it has been shown that compiler settings can be found that outperform standard `-Ox` switches for a single application, it is not known how to find such settings that work well for sets of applications. In this paper, we introduce a statistical technique to derive a methodology which trims down the search space considerably, thereby allowing a feasible and flexible solution for defining high performance optimization strategies. We show that our technique finds a single compiler setting for a collection of programs (SPECint95) that performs better than the standard `-Ox` settings of *gcc 3.3.1*.

Categories and Subject Descriptors

D.3.4 [**Programming Languages**]: Processors—*Code generation, Compilers, Optimization*

General Terms

Performance, Experimentation, Languages

Keywords

Compiler Switches, Statistical Analysis, Compiler Tuning, Back-end Optimization

1. INTRODUCTION

Modern compilers implement a host of back end optimizations. For example, *gcc 3.x* has over 50 compiler options [3]. However, very little is known about what all these options exactly do, how they interact, whether or not they enable or disable one another, etc. In order to deal with all these options, compiler writers usually define standard settings that can be used by programmers as `-Ox` switches. These switches contain those options that the compiler writer thinks are beneficial for many programs based on his experience. It is well known that for a single application better compiler settings can be found than those standard settings [2, 13]. However, it is not clear whether better settings can be found for a collection of applications. In this paper we show that this indeed is the case. We construct one single compiler setting that produces better optimized programs for a collection of widely differing applications than standard `-Ox` settings do, with up to 20% improvement. The collection of program we use in this study is the SPECint95 benchmark suite which is meant to be representative for a large collection of integer programs and hence has widely differing properties.

There exist some papers dealing with how to select options for individual programs based on statistical analysis of execution times resulting from compiling the program with (small) collections of settings [2, 13]. In this paper we focus on an extension of the problem of optimizing one single program, namely, how to find a compiler setting that optimizes many programs simultaneously. This setting should perform better than standard `-Ox` settings for most programs in the collection and equally well for the remaining programs. By this we mean that each program is optimized as well as the best performing `-Ox` switch and most programs even significantly better. This problem is relevant for example for supercomputing centers where a fixed collection of applications is used extensively and a single compiler optimization setting must be used for maintainability purposes. There are two main obstacles in finding these settings. First, since the number of different combinations of optimizations is exponentially large, the space, which has to be searched in order to find the optimal one, is huge. Second, the best sequence depends both on the application being compiled and on the target architecture.

The aim of our research is to develop a methodology which yields a feasible and near optimal solution for this problem. Our methodology is based on the simple idea that some compiler optimizations can positively influence each other and, therefore, should be turned on together. The fact that optimizations can interact is well known and expert compiler writers choose settings based on empirical experience of positive interaction. In this paper, we introduce a formal definition of the interaction between optimizations and present a quantitative model which allows us to measure these interactions.

The heuristic we propose in this paper is based on statis-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
CF'05, May 4–6, 2005, Ischia, Italy.
Copyright 2005 ACM 1-59593-019-1/05/0005...$5.00.

tical analysis of the effect of compiler options. This is one of the first papers to use statistical techniques to define compiler settings. We proceed by a three step approach. We choose this approach in order to severely reduce the number of candidate solutions. The steps are as follows. In the first step, we determine the set of options that have a large effect of their own and the set of pairs of options that positively interact. We then iteratively add single optimizations to the sets already obtained to get maximal sets of positively interacting options. In the second step, we try to combine the sets found in the previous step under the condition that these sets do not negatively influence each other. This weaker condition allows us to find compiler settings which have as many optimizations turned on as possible. Finally, in the third step, we test a small number of settings found in step two, selecting the setting with the best average improvement. All statistical analysis has been performed using a small but representative subset of the entire optimization space. This entails that the effects we measure are only approximations but it allows us to construct a small fragment of the entire search space beforehand and use the measured execution times freely during the subsequent analysis phases. Since our heuristic is intended to be used by compiler tuners in order to define a general setting usable for many programs in a given set, we can afford some profiling time in order to set up this initial search space. The ultimate goal of this research that we hope to address in future work is to find a strategy to find a compiler setting that is applicable to a specific application domain and that produces highly optimized code for that domain by incorporating domain specific knowledge.

The rest of this paper is organized as follows. Section 2 discusses our methodology to find a setting that is optimized for a collection of programs. Section 3 discusses the strategy to obtain such an optimized setting. Section 4 gives the results of our method on *gcc 3.3.1* and Section 5 gives an analysis of our methodology. Section 6 discusses related research. In section 7, we summarize this paper and discuss directions for future work.

2. A METHODOLOGY TO DEFINE A COMPILER SETTING

In this section, we introduce a systematic method to generate a compiler setting that is optimized for a collection of programs. We focus on the interaction between optimizations since this is an important issue in the determination of an optimal compiler setting. We believe that a nearly optimal compiler setting can be derived by combining optimizations which positively interact. Since there is no clear definition of the interaction between optimizations, we give a definition of interaction based the performance improvement of a collection of different optimization settings on a set of applications. We describe how execution times can be used to define the interaction between optimizations.

2.1 Detection of Interaction between Optimizations

In this section, we give the definition of interaction between compiler optimizations and we describe the procedure to construct subsets of optimizations that positively interact. We define the effect of compiler optimizations as the improvement expressed in terms of the real execution time of applications. The effect is dependent on the compiler setting used and therefore the effect is used to define interaction between these different compiler settings. In order to define the notion of interaction properly, we first need some notation.

We use a sequence of binary digits to describe a compiler setting s. The set of all possible sequences is the complete search space.

DEFINITION 2.1. *Let $n \geq 1$. Ω_n is the set of all sequences of n binary digits.*

$$\Omega_n = \{(a_1, \ldots, a_n) \ : \ a_i \in \{0, 1\}, 1 \leq i \leq n\}$$

The sequences in Ω_n express whether compiler options are on or off. If we denote the set of all available compiler options by $W = \{w_i \ : \ 1 \leq i \leq n\}$, then $a_i = 1$ in a sequence s means that optimization w_i is turned on and $a_i = 0$ means that optimization w_i is turned off.

Let s_{base} be the compiler setting without any optimization:

$$s_{base} = (0, \ldots, 0) \tag{1}$$

Let P be a set of benchmark programs and let $T(s, p)$ be the execution time for compiler setting $s \in \Omega_n$ and program $p \in P$. The effect of compiler setting $s \in \Omega_n$ on a program $p \in P$ is defined as a normalized function $e(s, p)$.

DEFINITION 2.2. *Let $s \in \Omega_n$ and let $p \in P$. We define the effect of compiler setting s on program p, denoted by $e(s, p)$, as follows.*

$$e(s, p) = \frac{T(s_{base}, p) - T(s, p)}{T(s_{base}, p)}$$

The value $e(s, p)$ describes how much the optimizations in s change the execution time of a non-optimized program p. The effect of a compiler setting s, denoted by $E(s)$, is defined as the average effect over all programs in P. We can simply take the average because $e(s, p)$ is normalized. In fact, $E(s)$ is the average of the improvements in performance of all the benchmark programs. Note that it can very well happen that a compiler setting generates substantial improvement on certain benchmark programs which are nullified by slow downs on other benchmark programs. In this case, the overall improvement is considered minimal. Also, $E(s)$ can have a negative value.

DEFINITION 2.3. *We define the effect of a compiler setting $s \in \Omega_n$ on a set of programs P, denoted by $E(s)$, as follows.*

$$E(s) = \frac{\sum_{p \in P} e(s, p)}{|P|}$$

Using these definitions, we define the interaction between two optimizations w_i and w_j as follows.

DEFINITION 2.4. *For each $1 \leq i, j \leq n$, $i \neq j$, we define*

$$s_i = (a_1, \ldots, a_n) \quad \textit{where } a_k = \begin{cases} 1 & : \ k = i \\ 0 & : \ \textit{otherwise} \end{cases}$$

$$s_{ij} = (a_1, \ldots, a_n) \quad \textit{where } a_k = \begin{cases} 1 & : \ k = i \ \textit{ or } \ k = j \\ 0 & : \ \textit{otherwise} \end{cases}$$

Given this notation, we define for each $w_i, w_j \in W$ the following interaction predicate I_1.

$$I_1(w_i, w_j) = \begin{cases} \textit{true} & : & E(s_{ij}) - E(s_i) > \textit{threshold, and} \\ & & E(s_{ij}) - E(s_j) > \textit{threshold} \\ \textit{false} & : & \textit{otherwise} \end{cases}$$

In other words, s_i is the compiler setting which turns on optimization w_i, s_j is the compiler setting which turns on optimization w_j, and s_{ij} is the compiler setting which turns on both w_i and w_j. The first inequality guarantees enough performance improvement if optimization w_j is added to w_i, and the second inequality checks if there is enough performance improvement if w_i is added to w_j. If $I_1(w_i, w_j) = \textit{true}$, then we conclude that w_i and w_j positively interact.

The problem with the above definition of interaction is that in practice the effect of one single optimization turned on can be minimal and the true effect of an optimization can only be estimated in the presence of other optimizations that are turned on. Therefore, we need a definition of interaction which allows several optimizations to be turned on simultaneously.

DEFINITION 2.5. *For $K \subseteq W$, $w_k \in W$, and $w_k \notin K$, we define*

$$s_K = (a_1, \ldots, a_n) \quad \textit{where} \quad a_i = \begin{cases} 1 & : & w_i \in K \\ 0 & : & \textit{otherwise} \end{cases}$$

$$s_{Kk} = (a_1, \ldots, a_n) \quad \textit{where} \quad a_i = \begin{cases} 1 & : & w_i \in K \ \textit{ or } \ i = k \\ 0 & : & \textit{otherwise} \end{cases}$$

We define the following interaction predicate I_2

$$I_2(K, w_k) = \begin{cases} \textit{true} & : & E(s_{Kk}) - E(s_K) > \textit{threshold} \\ \textit{false} & : & \textit{otherwise} \end{cases}$$

As we can see from this definition, the threshold in the inequality does not need to hold for $E(s_{Kk}) - E(s_k)$, making this definition of interaction different from the previous one. The reason for this is that this definition is used in an iterative algorithm which looks for successful grouping of optimizations by incrementally extending already successful groupings. Hence, the definition assumes that the effect of s_K is already significant and only for the addition of one optimization s_k it is investigated whether this leads to improved performance.

Although this definition of interaction is useful when searching for an optimal setting of compiler optimizations, the sheer number of possible optimization settings makes it impossible to apply this definition arbitrarily. For instance, in our experiments below we want to investigate the optimal setting for the GNU C-compiler *gcc 3.3.1* in which 54 compiler optimizations can be turned on or off. This yields $2^{54} \approx 10^{15}$ different possible settings. Hence using this definition arbitrarily would mean that 10^{15} different experiments would have to be conducted to find an optimal setting. Therefore, we want to trim down the possible number of different compiler settings to be investigated considerably. This is achieved in two ways: first, the proposed iterative search algorithm tries to use already successful settings which are incrementally extended. The second measure consists of trimming down the search space of possible settings. This is achieved by constructing a representative subset of the 2^{54} different possible settings using an orthogonal array, as explained in the next section.

$$\begin{pmatrix}
0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
1&0&1&0&1&0&1&0&1&0&1&0&1&0\\
0&1&1&0&0&1&1&0&0&1&1&0&0&1\\
1&1&0&0&1&1&0&0&1&1&0&0&1&1\\
0&0&0&1&1&1&1&0&0&0&0&1&1&1\\
1&0&1&1&0&1&0&0&1&0&1&1&0&1\\
0&1&1&1&1&0&0&0&0&1&1&1&1&0\\
1&1&0&1&0&0&1&0&1&1&0&1&0&0\\
0&0&0&0&0&0&0&1&1&1&1&1&1&1\\
1&0&1&0&1&0&1&1&0&1&0&1&0&1\\
0&1&1&0&0&1&1&1&1&0&0&1&1&0\\
1&1&0&0&1&1&0&1&0&0&1&1&0&0\\
0&0&0&1&1&1&1&1&1&1&0&0&0&0\\
1&0&1&1&0&1&0&1&0&1&0&0&1&0\\
0&1&1&1&1&0&0&1&1&0&0&0&0&1\\
1&1&0&1&0&0&1&1&0&0&1&0&1&1
\end{pmatrix}$$

Figure 1: Example of Orthogonal Array with 14 columns and 16 rows

2.2 Orthogonal Arrays

Orthogonal arrays have been used for planning experiments and they can be used to analyze factors in an experiment [6]. When an experiment has k factors or options, the total search space consists of 2^k settings. This total search space is called a *full factorial design*. It is impossible to examine a full factorial design and orthogonal arrays are one of the approaches to reduce this search space. Such a design is called a *fractional factorial design*. The data from the experiments generated with an orthogonal array contains enough meaningful information for factor analysis. More information about orthogonal arrays can be found in [6].

Briefly, an Orthogonal Array (OA) is a matrix of zeroes and ones. The rows of an orthogonal array represent experiments to be performed and the columns of the orthogonal array correspond to the different factors whose effects are being analyzed. For the purposes of this paper, an OA has the property that for two arbitrary columns, the patterns

$$0\,0 \qquad 0\,1 \qquad 1\,0 \qquad 1\,1$$

occur equally often. Figure 1 shows an example of an Orthogonal Array.

An OA has the property that any option is turned on and off equally often in the experiments defined by the rows of the OA. Moreover, for the rows that turn a certain option on, any other option is turned on and off equally often as well. These properties allow us to calculate the main effects of options accurately using an OA without the need to use a full factorial design.

In this paper, we do not use OAs to calculate the effect of options. In contrast, we exploit the fact that a (sufficiently large) OA is a good representative fragment of the entire search space. Hence, we use one particular OA and we measure the execution times of all programs in our set P using the rows as compiler settings. Then we use these measured execution times to approximate the effect of compiler settings and the interaction predicate, as explained in the next section.

2.3 Defining Interaction Using a Representative Subset of the Search Space

Having defined a representative subset S of the complete search space by means of an Orthogonal Array, our algorithm is based on using the measured performance results

of the compiler settings which are represented by S. This means that initially all the compiler settings represented by S are executed on a specific platform for all the benchmarks considered. The execution times are stored in a database to be used by our methodology and no other execution times are used. This means that the experimental results needed to verify previously given definitions of interaction might not be available. For instance, in Definition 2.5, the compiler settings s_K, or s_k, or s_{Kk} might not be part of the subset S. Therefore, we need to adopt the previously given definitions so that only experimental results derived from S are necessary to establish interaction or not.

DEFINITION 2.6. *Let $S \subseteq \Omega_n$ and let $A, B \subseteq W$ with $A \cap B = \varnothing$. We define*

$$S^{(A=1)(B=0)} = \{s \ : \ s \in S \text{ with } a_i = 1 \text{ for } w_i \in A, \text{ and with } a_i = 0 \text{ for } w_i \in B\}$$

$S^{(A=1)(B=0)}$ is a (possibly empty) subset of S, in which the optimizations in A are turned on and the optimizations in B are turned off. Below we write $S^{(A=1)}$ in case $B = \varnothing$. Note that $A \cup B$ is not necessarily equal to the complete set W. Optimizations that do not appear in either A or B may be turned on or off arbitrarily. The approximate effect of $S^{(A=1)(B=0)}$ is described as follows.

DEFINITION 2.7. *Let $S \subseteq \Omega_n$ and let $A, B \subseteq W$ with $A \cap B = \varnothing$. The* approximate effect *of $S^{(A=1)(B=0)}$ is defined as*

$$E\left(S^{(A=1)(B=0)}\right) = \begin{cases} \frac{\sum_{s \in S^{(A=1)(B=0)}} E(s)}{|S^{(A=1)(B=0)}|} & : \ S^{(A=1)(B=0)} \neq \varnothing \\ 0 & : \ \textit{otherwise} \end{cases}$$

The approximate effect is applied to Definition 2.5 to obtain a new definition of interaction.

DEFINITION 2.8. *Let be $K \subseteq W$ and let $w_k \in W$ such that $w_k \notin K$. We define the interaction between K and w_k as follows.*

$$I(K, w_k) = \begin{cases} \textit{true} & : \ E\left(S^{(K \cup \{w_k\}=1)}\right) \\ & \quad - E\left(S^{(K=1)(\{w_k\}=0)}\right) > \textit{threshold} \\ \textit{false} & : \ \textit{otherwise} \end{cases}$$

The set $S^{(K=1)}$ is split into two parts, $S^{(K \cup \{w_k\}=1)}$ and $S^{(K=1)(\{w_k\}=0)}$ depending on the state of w_k. The first part $S^{(K \cup \{w_k\}=1)}$ consists of the compiler settings that turn on the optimizations in K as well as w_k. The second part $S^{(K=1)(\{w_k\}=0)}$ consists of compiler settings that turn on the optimizations in K and turn off optimization w_k.

3. ALGORITHM TO FIND A COMPILER SETTING

In this section we discuss the three steps of our algorithm to find a compiler setting that is highly optimized for a set of applications simultaneously.

3.1 Step 1: Finding Maximal Subsets of Positively Interacting Options

Using Definition 2.8, we construct an algorithm to produce subsets of optimizations that positively interact in two phases. This is done iteratively by taking an initial choice of single optimizations which have a relatively large effect in the first phase. Then, in each iteration, the subsets of positively interacting optimizations are being extended with optimizations which enforce the previous subsets in the second phase. At each step of the algorithm, the set C consists of subsets of optimizations that positively interact.

In the first phase, the initial set C_1 is selected as the collection of single optimizations that have a relatively large effect. We calculate the effect of each individual optimizations with Definition 2.7 as follows. For each i, $A = \{w_i\}$ and Definition 2.7 is applied with $B = \varnothing$. So we compute the average improvement of all optimization settings in S in which w_i is turned on. Then we rank all optimizations w_i according to their largest average effect. C_1 is then constructed by taking the top M optimizations of this list. In our implementation we have heuristically chosen M to be 10, so the algorithm starts with a subset C_1 which consists of 10 individual optimizations.

After this first phase in which a collection of individual optimizations are selected which have a significant effect, in the second phase of Step 1 we select from the remaining optimization those optimizations which are successful if they are combined with another optimization. The reason for this is that certain optimizations are only successful if they are enabled by another optimization. If the start set of successful optimizations would only consist of individual optimizations, the optimizations which are successful only when combined with an (enabling) optimization would be left out. For defining the second phase of Step 1 of the algorithm we need the notion of 'left out' optimizations W^k.

DEFINITION 3.1. *W^k is the set of optimizations that do not appear in the elements of $C_1, \ldots, C_{k-1}$.*

$$W^k = \{w \in W \ : \ \forall 1 \leq i \leq k-1 \ . \ \forall c \in C_i \ . \ w \notin c\}$$

Therefore, W^2 is the set of those optimizations that are left over from the construction of C_1. After the construction of C_1, each optimization in W is combined with all the optimizations in W^2. Each combination is examined with Definition 2.8 to form potential subsets of C_2. The combination of two optimizations in C_1 is not investigated because we want to find two optimizations that enable each other while they have little effect when used on their own. If one of the optimizations which is used in the combinations in C_2 appears in C_1, the optimization is removed from C_1 since the optimization is considered to work better when combined with another combination.

After C_2 has been found, C_i is inductively constructed by adding one optimization from the set of optimizations W^i to one of the elements in C_{i-1} when it satisfies the predicate I given in Definition 2.8. The formal algorithm is given in Algorithm 1. For each iteration to construct C_i, Algorithm 1 removes elements from C_{i-1}. This procedure is based on the concept that the extended combination may have better improvement than the original one according to Definition 2.8. The algorithm produces C which consists of all C_i, that is, $C = \bigcup_i C_i$. Each C_i consists of sets of i positively interacting optimizations. The algorithm stops if no set can be

extended anymore. Hence we construct sets of maximal sequences of options.

Algorithm 1

```
C ← ∅
Create C_1
C ← C ∪ C_1
Create W^2
C_2 ← ∅
for all k ∈ W do
  for all w ∈ W^2 do
    if I({k}, w) is true then
      C_2 ← C_2 ∪ {{w, k}}
      if k ∈ C_1 then
        C_1 ← C_1 − {k}
      end if
    end if
  end for
end for
C ← C ∪ C_2
i ← 2
repeat
  i ← i + 1
  C_i ← ∅
  Create W^i
  V ← C_{i−1}
  for all K ∈ V do
    for all w ∈ W^i do
      if I(K, {w}) is true then
        C_i ← C_i ∪ {K ∪ {w}}
      end if
    end for
    if there exists c ∈ C_i with K ⊂ c then
      C_{i−1} ← C_{i−1} − {K}
    end if
  end for
  C ← C ∪ C_i
until C_i = ∅ or W^i = ∅
```

3.2 Step 2: Combining Subsets

Up till now we described how successful subsets of optimizations can be identified. Because the goal of this paper is to define a compiler setting which works well for a set of different applications, in this phase of the algorithm we identify combinations of these subsets which do not negatively interfere with each other. So instead of looking for optimizations which enforce each other, we now look at sets of optimizations which do not have a negative impact on each other.

The algorithm for constructing these compiler settings uses an evaluation predicate again. This predicate decides whether two subsets must be combined.

DEFINITION 3.2. *Let $K_1, K_2 \subset W$ and $K_1 \neq K_2$. We define the approximate interaction between K_1 and K_2 as follows.*

$$I'(K_1, K_2) = \begin{cases} \textit{true} & : \; E\left(S^{(K_1 \cup K_2 = 1)}\right) - E\left(S^{(K_1 = 1)}\right) \geq 0 \\ & \quad \textit{and} \\ & \; E\left(S^{(K_1 \cup K_2 = 1)}\right) - E\left(S^{(K_2 = 1)}\right) \geq 0 \\ \textit{false} & : \; \textit{otherwise} \end{cases}$$

The first equation compares the effect of the optimizations in K_1 and K_2 with the effect of the optimizations in K_1. The second equation compares the effect of the optimizations in K_1 and K_2 with the effect of the optimizations in K_2. As can be seen from this definition, no threshold value is used in the inequality indicating that turning on the optimization represented by K_2 in addition to the optimizations represented by K_1 does not lead to a performance degradation and visa versa.

Algorithm 2

```
C'_1 ← C
i ← 1
repeat
  i ← i + 1
  V ← C'_{i−1}
  C'_i ← ∅
  for all K_1 ∈ V do
    for all K_2 ∈ C do
      if K_1 ≠ K_2 then
        if I'(K_1, K_2) is true then
          C'_i ← C'_i ∪ {K_1 ∪ K_2}
          C'_{i−1} ← C'_{i−1} − {K_1}
        end if
      end if
    end for
  end for
until C'_i = ∅
```

The algorithm now investigates which combinations of subsets in C can be combined. It starts with all the elements in C and tries to combine them. K_2 in the algorithm is always one of the elements of C, and the algorithm stops when a set C'_i is empty, that is, when there are no sets left that can be combined and yield a higher improvement. This algorithm produces $C'_1, C'_2, \ldots$ that contain subsets of optimizations having various numbers of optimizations turned on. For convenience sake, we classify them according to the number of optimizations turned on and store them in S_i, where i denotes the number of optimizations. The algorithm is given in Algorithm 2. Again, the algorithm stops when sets cannot be extended anymore.

3.3 Step 3: Selecting the Best Setting

We have generated candidate compiler settings in $\mathcal{S} = \bigcup_i S_i$ in the previous two steps generated based on an analysis of the reduced search space. Since there is no exact information about the compiler settings we generated, we examine them again in this step and identify one setting from them as our compiler setting. Since it is not feasible to execute all settings in S to select the overall best one, we restrict our search space again. Because we are looking for a most versatile compiler setting which performs well on a number of different applications, a setting is selected

defer-pop	force-mem
force-addr	omit-frame-pointer
optimize-sibling-calls	inline
inline-functions	keep-static-consts
merge-constants	merge-all-constants
branch-count-reg	function-cse
strength-reduce	thread-jumps
cse-follow-jumps	cse-skip-blocks
rerun-cse-after-loop	rerun-loop-opt
gcse	gcse-lm
gcse-sm	loop-optimize
crossjumping	if-conversion
if-conversion2	delete-null-pointer-checks
expensive-optimizations	optimize-register-move
delayed-branch	schedule-insns
schedule-insns2	sched-interblock
sched-spec	sched-spec-load
sched-spec-load-dangerous	caller-saves
move-all-movables	reduce-all-givs
peephole	reorder-blocks
reorder-functions	strict-aliasing
align-functions	align-labels
align-loops	align-jumps
rename-registers	cprop-registers
float-store	single-precision-constant

Table 1: Options of gcc 3.3.1 used.

force-mem
omit-frame-pointer
optimize-sibling-calls
inline
inline-functions
keep-static-consts
function-cse
thread-jumps
gcse
align-labels

Table 2: Initial Optimizations in C_1

from those settings which have most optimizations turned on. Starting from the set S_i with the largest settings (i largest in the collection of settings found in the Step 2), only a limited number of sets S_k with $k < i$ are considered. In our case study, at most 50 of the largest compiler settings are considered. After execution, we calculate $E(s)$ for each setting s chosen. We choose the compiler setting s as our setting when $E(s)$ is maximal.

4. RESULTS

We have evaluated our methodology on the *gcc 3.3.1* compiler that has 54 compiler optimizations implemented, and we use 50 optimizations that are not experimental according to the on-line documentation [3]. These options are given in Table 1. An orthogonal array with 50 columns and 400 rows is used as the search space S. The compiler settings in S are executed with the seven benchmark programs from the *SPECint95* suite. We did not use `m88ksim` because it did not compile correctly for all settings on our platform. We have used a Pentium 4 at 2.8GHz as the architecture in this experiment.

We use 10 optimizations out of 50 optimizations for the initial set C_1. The 10 optimizations are selected as those optimizations that have a large effect according to Definition 2.7. The optimizations are shown in Table 2.

Table 3 shows the number of elements of the set C_i generated by Algorithm 1. We have applied several threshold values to see which value is most adequate: the values 0.004, 0.005, 0.006, and 0.007 have been used for this experiment. A threshold value of 0.006 tends to construct a large number of subsets. This is because a small threshold leads to many combinations in the early steps in the algorithm. For instance, a threshold value of 0.004 has 72 elements in C_2. When there are many elements in C_k, there are few candidates in W^{k+1} since W^{k+1} consists of the optimizations that do not appear in C_k This prevents us to construct many subsets of optimizations. Hence, we choose 0.006 as the threshold value in this case study.

Threshold	C_1	C_2	C_3	C_4	C_5	C_6
0.004	3	72	-	-	-	-
0.005	6	14	12	3	2	-
0.006	7	5	2	1	12	6
0.007	10	3	-	-	-	-

Table 3: Number of partial sets in C

S_6	S_7	S_8	S_9	S_{10}	S_{11}	S_{12}
1	6	25	62	142	233	320

S_{13}	S_{14}	S_{15}	S_{16}	S_{17}	S_{18}	S_{19}
342	259	162	141	100	41	4

Table 4: Number of Compiler Settings in $\mathcal{S}$

Next, we apply Algorithm 2 to the results for a threshold value of 0.006 in Table 3. The number of compiler settings generated by this algorithm is shown in Table 4.

There are sets S_i for $6 \leq i \leq 19$ in $\mathcal{S}$ and we choose the settings in S_{18} and S_{19} to execute in step 3 of our algorithm. Since the total number of elements in S_{18} and S_{19} is 45, it is feasible to execute all compiler settings in these sets. The setting which has the best improvement is chosen as our setting, denoted by s_{new}. In this case study, a setting in S_{18} has the best improvement. The setting turns on the 18 optimizations given in Table 5 and turns off the others. For comparison, option `-O1` defined by *gcc* uses 10 optimizations, option `-O2` uses 36 optimizations, and option `-O3` uses 38 optimizations.

Figure 2 shows the performance improvement of the setting generated by using our methodology, and the performance improvement of the option `-O1`, `-O2`, and `-O3`. The improvement of the generated optimization setting s_{new} is

omit-frame-pointer
inline
keep-static-consts
merge-all-constants
branch-count-reg
strength-reduce
cse-follow-jumps
rerun-cse-after-loop
gcse-lm
if-conversion2
delayed-branch
sched-interblock
sched-spec-load-dangerous
reduce-all-givs
peephole
reorder-blocks
reorder-functions
align-labels

Table 5: Setting found by our methodology

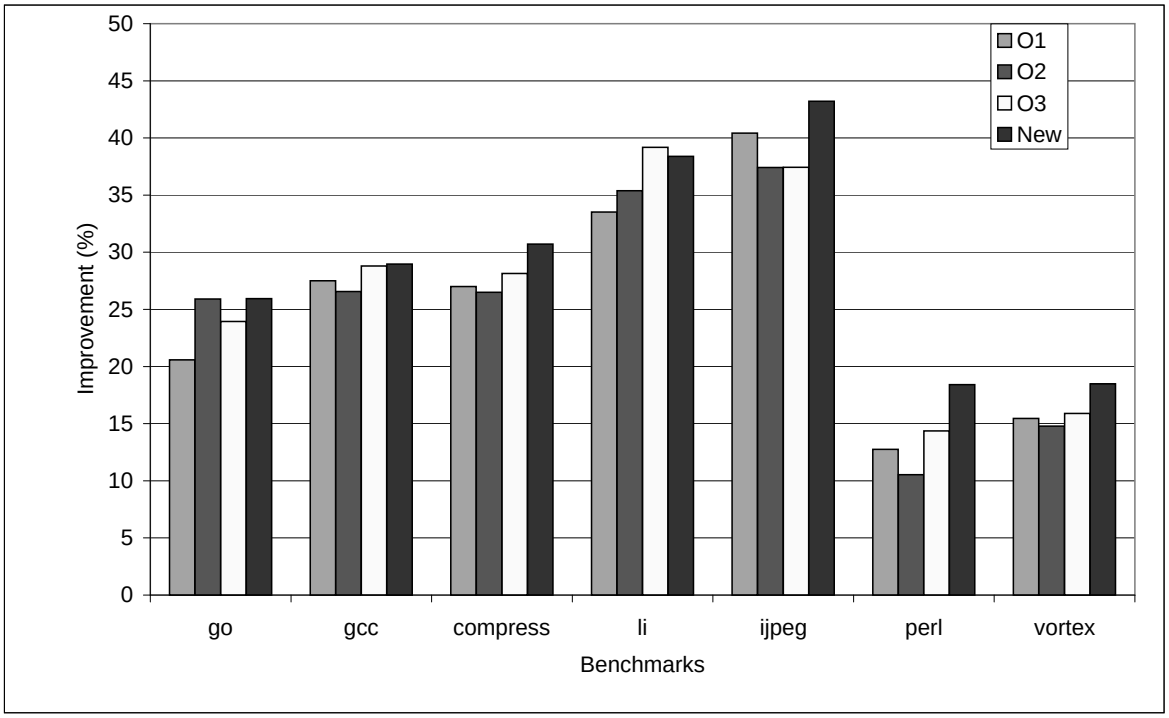

Figure 2: Improvements of New Setting and `-Ox` Options

calculated as $E(s_{new})$. A first observation we make is that in several cases, `-O1` performs better than `-O2`. For `ijpeg`, it is even the case that `-O1` performs better than both `-O2` and `-O3`.

The generated compiler setting, denoted by *new* in Figure 2, performs better than the `-O1`, `-O2` and `-O3` switches for all benchmark programs we used, except for `li` when optimized with `-O3` which performs slightly better than our setting s_{new} (39.2% vs. 38.4%). In particular, *new* delivers the best performance for `perl`, giving almost twice as much improvement as `-O2` (18.4% vs. 10.5%). This shows that our methodology is capable of finding a compiler setting for a collection of programs that behave quite differently and would require different compiler settings when tuned individually.

5. OBSERVATIONS

In section 2, we have proposed our methodology to define a compiler setting. In that section, Algorithm 1 and Algorithm 2 have been given. Algorithm 1 uses a threshold value in the evaluation predicate given in Definition 2.8. Several threshold values, 0.004, 0.005, 0.006, and 0.007, have been studied in the case study in Section 4. We have chosen a threshold value which produces the most subsets. If there are several threshold values that produce subsets that have the same maximum number of optimizations turned on, the threshold value that produces the largest number of subsets must been chosen.

In Table 3, we can also see that the results are different for each threshold value. A value of 0.006 has the most subsets that positively interact, but there are few subsets produced with a value of 0.007. This means that the algorithm is very sensitive to the threshold value. Therefore, it is necessary to choose this value carefully. Since the search space is already fixed beforehand, it is possible to have some trials of the algorithm to choose the most suitable threshold value which produces the most subsets.

The algorithm incrementally adds one optimization to an already derived set and checks whether this addition is significant. Note that it could be possible to find another set of, say, 2 (or more) optimizations which, combined with the previous set, makes all optimizations significant. This means that it can be the case that, for instance,

$$I(\{w_1, w_4, w_5\}, w_2) = 0 \quad \text{and}$$
$$I(\{w_1, w_4, w_5\}, w_6) = 0$$

which means that neither $\{w_1, w_2, w_4, w_5\}$ nor $\{w_1, w_4, w_5, w_6\}$ is in S_4. However, it could be the case that

$$I(\{w_1, w_4, w_5, w_6\}, w_2) = 1 \quad \text{or}$$
$$I(\{w_1, w_2, w_4, w_5\}, w_6) = 1 \quad \text{or}$$
$$I(\{w_2, w_4, w_5, w_6\}, w_1) = 1 \quad \text{or}$$
$$I(\{w_1, w_2, w_5, w_6\}, w_4) = 1 \quad \text{or}$$
$$I(\{w_1, w_2, w_4, w_6\}, w_5) = 1$$

By construction, however, $\{w_1, w_2, w_4, w_5, w_6\}$ will not be in S_5 while it is a significant set. However, the likelihood of the previously sketched situation to occur is low and therefore we do not consider these cases in our methodology. In a certain sense, the fact that the combination w_2 and w_6 interacts with $\{w_1, w_4, w_5\}$ is a higher order effect that we do not consider in this paper. The other reason is that the

proposed algorithm would grow exponentially in compute time.

In Algorithm 1 and Algorithm 2, the approximate effect of compiler optimizations defined in Definition 2.7 is used. The approximate effect is calculated by using execution times using an Orthogonal Array. From the definition of the effect, at least one execution result satisfying Definition 2.7 is necessary to calculate the effect. It is possible that there is no execution result which satisfies Definition 2.7 due to the reduced search space. When the algorithms encounter this situation, they regard the optimizations as non-interacting optimizations, and do not use them.

6. RELATED WORK

Although compiler optimizations have been proposed since the invention of the compiler, there has not been much work carried out to study which optimizations should be turned on for which application. The Dragon book [1], Waite and Goos [17], or Tremblay and Sorenson [15] simply give a collection of possible backend optimizations without much discussion about when to use them. Muchnick [11] simply sums up which optimizations should be employed one after the other based on experience.

A number of approaches to select best optimizations have been proposed by searching the optimization space. Iterative compilation [7] or the GAPS system that employs genetic algorithms [12] search for source level transformations. [4] use genetic algorithms to find optimal low level code sequences, paying attention to both performance and code size. [10] use machine learning techniques based on oblique decision trees to find compiler heuristics specific for a particular platform. [8] applies instance-based learning techniques to optimize JAVA programs for a particular platform. [16] discusses an implementation of iterative compilation in the Intel IA-64 production compiler. In contrast to these efforts, our approach uses statistical analysis to systematically prune the search space and is focused on compiler switches. [14] uses an evolutionary algorithm to automatically find effective compiler heuristics for a particular application by searching for priority functions that drive optimizations. This work, however, does not address the general problem of setting compiler switches.

Granston and Holler [5] propose a tool for automatic selection of compiler options, called *Dr. Options*. This tool uses information about the application supplied by the user and a set of tuning rules that have been created by interviewing tuning experts and analyzing optimization experiments. Hence, much compiler and application specific knowledge must be collected in order to create or even use such a tool. In contrast, our approach uses little knowledge but statistical analysis instead to find a setting. VISTA [20] is an interactive tool to assist the application programmer in finding optimizations and their phase order. However, it does not provide automatic optimization selection like the present work. [19] provides a framework for predicting the impact of loop transformations to assist in selecting the optimal one. However, it is not clear how backend transformations can be incorporated in this framework and it does not provide automatic selection facilities. Whitfield and Soffa [18] propose a framework for specifying transformations and an automatic optimizer generator in order to experiment with transformations. However, this does not solve the problem of which transformations to enable.

Chow and Wu [2] and Pinkers et al. [13] employ statistical techniques to decide which compiler options to set for one application. [2] employs linear regression analysis. [13] uses Orthogonal Arrays to determine the main effect of options [9] and set those options with a high positive main effect. They then apply their procedure iteratively on the remaining options to decide which ones of those improve the program more. Both of these papers focus on optimizing a single application. It is clear that these approaches can also be used for collections of programs by minimizing total execution time or maximizing average improvement. However, this does not necessarily solve the problem we set out to solve in this paper, namely, that each program is optimized as least as well as the best `-Ox` switch and most of them better. In contrast, we have shown that our approach indeed does solve this problem for collections of programs with widely differing behavior, in our case, the SPECint95 benchmark suite.

7. CONCLUSION

This paper introduced a systematic method to generate a compiler setting that is optimized for a collection of applications. First, we detect several subsets of compiler optimizations that positively interact. Next, we combine these sets to yield complete compiler settings. Since the method can generate several compiler settings, we choose one of them by profiling them and selecting the one with best average improvement.

We have applied our method to *gcc 3.3.1*, which has a large number of compiler options, to examine the efficiency of our method. By using 400 compiler settings generated by an orthogonal array as a reduced search space, we have obtained real execution times for the SPECint95 benchmark suite which is a collection of programs with widely differing behavior. This reduced search space has been used to make all decisions in the algorithm. Our methodology produced a compiler setting that gives better performance than the standard `-O1`, `-O2`, and `-O3` options provided by *gcc*.

Our methodology enables us to define an optimized compiler setting without any knowledge of the available compiler options. Therefore, our methodology can be useful to determine a compiler setting for an arbitrary architecture, or for an arbitrary set of applications. In future work, we want to address the problem of finding settings for a particular application domain, by also incorporating domain specific knowledge in the selection of relevant options. By using another measure for the effect of compiler options than execution time, this method can be used to find good settings for less energy consumption or for smaller code size.

8. REFERENCES

[1] A.V. Aho, R. Sethi, and J.D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 1986.

[2] K. Chow and Y. Wu. Feedback-directed selection and characterization of compiler optimizations. In *Proc. 2nd Workshop on Feedback Directed Optimization*, 1999.

[3] GNU Consortium. GCC online documentation. `http://gcc.gnu.org/onlinedocs/`.

[4] K.D. Cooper, P.J. Schielke, and D. Subramanian. Optimizing for reduced code space using genetic

algorithms. In *Proc. Languages, Compilers, and Tools for Embedded Systems (LCTES)*, pages 1–9, 1999.

[5] E. Granston and A. Holler. Automatic recommendation of compiler options. In *Proc. 4th Workshop on Feedback-Directed and Dynamic Optimization*, 2001.

[6] A.S Hedayat, N.J.A Sloane, and J. Stufken. *Orthogonal Arrays: Theory and Applications*. Series in Statistics. Springer Verlag, 1999.

[7] T. Kisuki, P.M.W. Knijnenburg, and M.F.P. O'Boyle. Combined selection of tile sizes and unroll factors using iterative compilation. In *Proc. PACT*, pages 237–246, 2000.

[8] S. Long and M.F.P. O'Boyle. Adaptive JAVA optimisation using instance-based learning. In *Proc. ICS*, pages 237–246, 2004.

[9] R.L. Mason, R.F. Gunst, and J.L. Hess. *Statistical Design and Analysis of Experiments*. Series in Probability and Statistics. John Wiley and Sons, 2003.

[10] A. Monsifrot, F. Bodin, and R. Quiniou. A machine learning approach to automatic production of compiler heuristics. In *Proc. AIMSA*, LNCS 2443, pages 41–50, 2002.

[11] S.S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.

[12] A. Nisbet. GAPS: Genetic algorithm optimised parallelization. In *Proc. Workshop on Profile and Feedback Directed Compilation*, 1998.

[13] R.P.J. Pinkers, P.M.W. Knijnenburg, M. Haneda, and H.A.G. Wijshoff. Analysis of compiler options using orthogonal arrays. In *Proc. CPC*, pages 137–148, 2004.

[14] M. Stephenson, M. Martin, and U.M. O'Reilly. Meta optimization: Improving compiler heuristics with machine learning. In *Proc. PLDI*, pages 77–90, 2003.

[15] J.P. Tremblay and P.G Sorenson. *The Theory and Practice of Compiler Writing*. McGraw-Hill, 1985.

[16] S. Triantafyllis, M. Vachharajani, N. Vachharajani, and D.I. August. Compiler optimization-space exploration. In *Proc. International Symposium on Code Generation and Optimization*, pages 204–215, 2003.

[17] W.M. Waite and G. Goos. *Compiler Construction*. Springer Verlag, 1984.

[18] D.L. Whitfield and M.L. Soffa. An approach for exploring code improving transformations. *ACM Transactions on Programming Languages and Systems*, 19(6):1053–1084, 1997.

[19] M. Zhao, B. Childers, and M.L. Soffa. Predicting the impact of optimizations for embedded systems. In *Proc. LCTES*, pages 1–11, 2003.

[20] W. Zhao, B. Cai, D. Whalley, M.W. Bailey, R. van Engelen, X. Yuan, J.D. Hiser, J.W. Davidson, and K. Gallivan. VISTA: A system for interactive code improvement. In *Proc. LCTES*, pages 155–164, 2002.